

---

**Équipe 09**

---

**PlaniSup**  
**Document d'architecture logicielle**

**Version 1.1**

## Historique des révisions

| Date       | Version | Description                                   | Auteur    |
|------------|---------|-----------------------------------------------|-----------|
| 2025-10-01 | 1.0     | Version initiale du document                  | Équipe 09 |
| 2025-10-02 | 1.1     | Correction du document selon les commentaires | Équipe 09 |
|            |         |                                               |           |
|            |         |                                               |           |
|            |         |                                               |           |

# Table des matières

|                                                 |    |
|-------------------------------------------------|----|
| 1. Introduction.....                            | 4  |
| 2. Objectifs et contraintes architecturaux..... | 4  |
| 3. Vue des cas d'utilisation.....               | 6  |
| 4. Vue logique.....                             | 6  |
| 5. Vue des processus.....                       | 9  |
| 6. Vue de déploiement.....                      | 15 |
| 7. Taille et performance.....                   | 16 |

# Document d'architecture logicielle

## 1. Introduction

Ce document présente l'architecture logicielle de PlaniSup, une application web destinée à la gestion et à la validation des plans d'études aux cycles supérieurs de Polytechnique Montréal. Il vise à fournir une vue d'ensemble complète de la structure du système, de ses composants principaux et de leurs interactions. L'architecture décrite dans ce document répond aux exigences définies dans le SRS, tout en tenant compte des contraintes technologiques et institutionnelles de Polytechnique Montréal. Ce document sert de référence principale pour l'équipe de développement et les parties prenantes du projet. Il guide la mise en œuvre du système et facilite la prise de décisions techniques cohérentes tout au long du cycle de développement. Le document est organisé en plusieurs sections, chacune offrant une perspective différente sur l'architecture du système. Il commence par les objectifs et contraintes architecturaux qui définissent le cadre des décisions de conception. Ensuite la vue des cas d'utilisation présente les scénarios d'utilisation principaux. La vue logique décrit la structure en termes de paquetages et de leurs relations. La vue des processus illustre les interactions entre composants lors de l'exécution. La vue de déploiement détaille la configuration matérielle et logicielle nécessaire. Enfin, la section taille et performance aborde les considérations qui ont influencé les choix architecturaux.

## 2. Objectifs et contraintes architecturaux

Cette section présente les objectifs et contraintes ayant un impact direct sur l'architecture du système PlaniSup. Ces éléments influencent les choix technologiques, les modèles de conception ainsi que les décisions d'implémentation. Les solutions envisagées pour y répondre sont également détaillées.

### 2.1. Adaptation des interfaces utilisateurs

L'application doit s'adapter à six profils utilisateurs distincts (étudiant, directeur, agent administratif, coordonnateur, registraire et administrateur), chacun ayant des besoins spécifiques et des niveaux d'expérience variés. Les interfaces doivent être intuitives, accessibles depuis tous les navigateurs et s'afficher correctement. Pour répondre à ces exigences, le cadriciel Angular sera utilisé et couplé à Angular Material pour offrir une interface réactive et cohérente. Des composants modulaires et réutilisables seront développés pour personnaliser les vues en fonction du rôle de l'utilisateur tout en maintenant une expérience utilisateur uniforme et conforme aux normes d'utilisabilité définies dans le SRS.

### 2.2. Gestion des rôles des utilisateurs

La sécurité des données académiques et la restriction d'accès selon le profil utilisateur sont des impératifs critiques. Le système doit différencier automatiquement les comptes employés (code permanent commençant par « p ») des comptes étudiants et attribuer les permissions appropriées. L'authentification centralisée via le service CAS de Polytechnique sera intégrée pour garantir que seuls les utilisateurs autorisés accèdent aux fonctionnalités correspondant à leur rôle. Un mécanisme de gestion des rôles (RBAC) sera implémenté côté serveur pour valider systématiquement les droits d'accès aux données et aux fonctionnalités conformément aux politiques de sécurité de l'établissement.

### 2.3. Validation dynamique et en temps réel des règles académiques

Le cœur métier de PlaniSup repose sur la validation en temps réel des plans d'études selon des règles académiques complexes et évolutives (crédits, prérequis, compatibilités de sessions, etc.). Ces règles doivent être modifiables via

l'interface d'administration sans intervention technique. Les règles seront modélisées dans un format structuré et versionable (ex. JSON) interprétable par un moteur de validation dédié. Ce moteur sera invoqué à la fois côté client pour une rétroaction immédiate lors de l'édition du plan et côté serveur pour la validation finale. La configuration des règles, stockée en base de données, permettra une maintenance aisée et une adaptation rapide aux changements des programmes.

## **2.4. Gestion du workflow d'approbation multi-étapes**

Le processus d'approbation des plans d'études suit une séquence précise qui varie selon le type de programme. Pour les programmes de DESS et maîtrise professionnelle, le workflow est : étudiant → agent administratif → coordonnateur → registrariat. Pour tous les autres programmes (maîtrise recherche, doctorat), le workflow est : étudiant → directeur → agent administratif → coordonnateur → registrariat. Cette différence fondamentale doit être gérée dynamiquement par le système. Chaque étape doit être traçable et notifiée aux acteurs concernés. Le système doit permettre l'annulation d'un plan soumis et conserver l'historique complet des approbations, des rejets et des commentaires associés. Ces workflows sont modélisés sous forme de machines à états configurables avec historique des transitions. L'historique des validations, refus et commentaires sera conservé pour assurer la transparence et la traçabilité requise par le processus académique tout en s'adaptant automatiquement au type de programme de l'étudiant.

## **2.5. Intégration, déploiement et conformité institutionnelle**

L'application sera hébergée sur les serveurs de Polytechnique Montréal et devra respecter leurs contraintes techniques et de sécurité. Le déploiement s'appuiera sur une infrastructure containerisée avec Docker pour garantir la portabilité entre les environnements. Un pipeline CI/CD via BitBucket assurera l'intégration continue et le déploiement automatisé sous réserve de validation par le Service informatique. L'authentification via CAS, le chiffrement HTTPS et le respect des politiques de sécurité et d'infrastructure de Polytechnique seront intégrés dès la conception pour assurer la conformité institutionnelle.

## **2.6 Performance, scalabilité et disponibilité**

Bien que le nombre d'utilisateurs simultanés soit estimé à 500, l'application doit maintenir des temps de réponse inférieur à 2 secondes pour 95 % des actions et supporter les pics d'utilisation lors des périodes de soumission. Une architecture modulaire (frontend Angular, backend Node.js/Express, base MongoDB) sera adoptée pour favoriser la scalabilité. La mise en cache des données fréquemment consultées (catalogues de cours, règles de validation) et l'optimisation des requêtes database permettront de respecter les exigences de performance tout en garantissant une disponibilité de 99,9 % et un temps de rétablissement inférieur à 6 heures en cas de panne.

### 3. Vue des cas d'utilisation

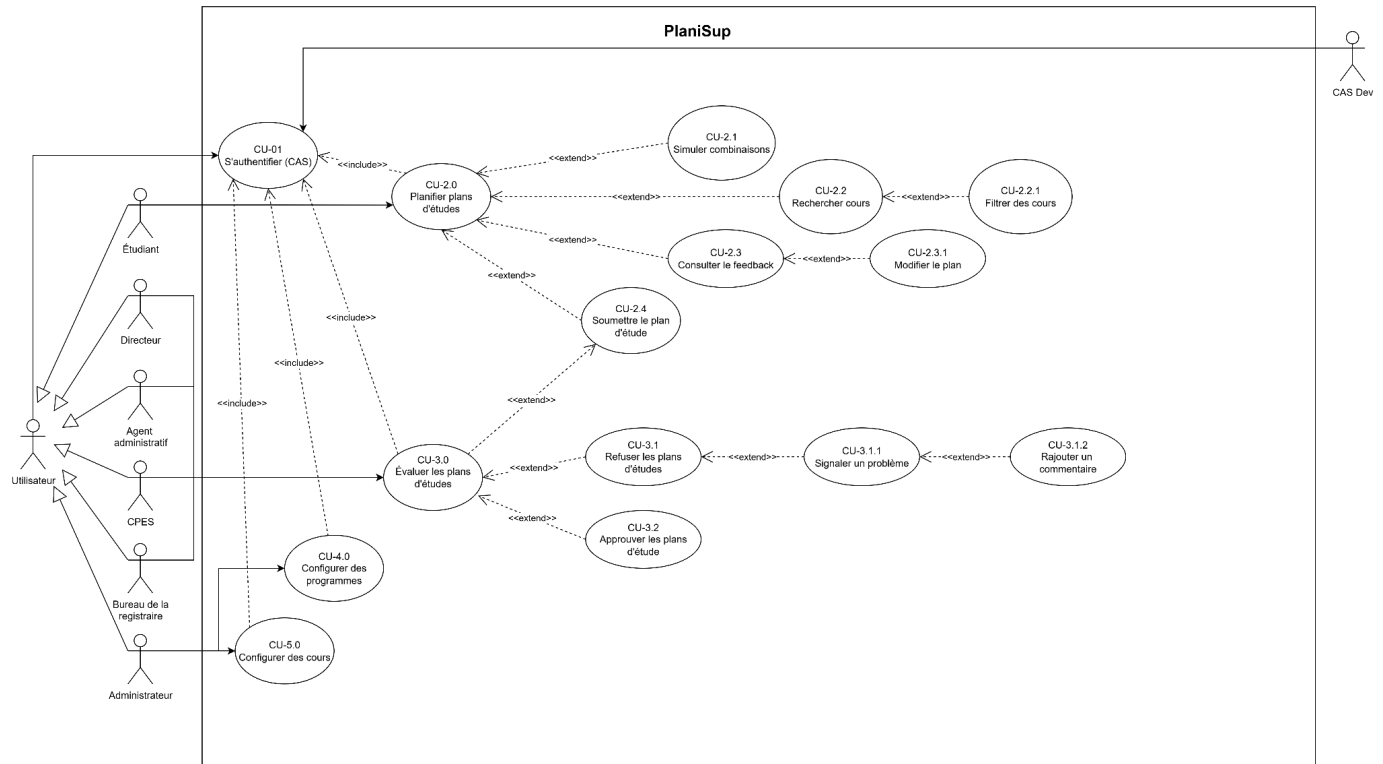


Figure 3.1 – Diagramme de cas d'utilisation de PlaniSup

### 4. Vue logique

Cette section présente la vue logique de l'application PlaniSup. L'architecture de l'application sera décrite sous forme de paquetages. Il y aura un diagramme de paquetages qui représentera le front-end et le serveur. Les trois paquetages seront ensuite mis en interaction afin de présenter le comportement global de l'application.

| Paquetage | Fonction                                       |
|-----------|------------------------------------------------|
| Front-End | Se charge des fonctionnalités de l'application |
| Serveur   | Se charge des fonctionnalités du serveur       |

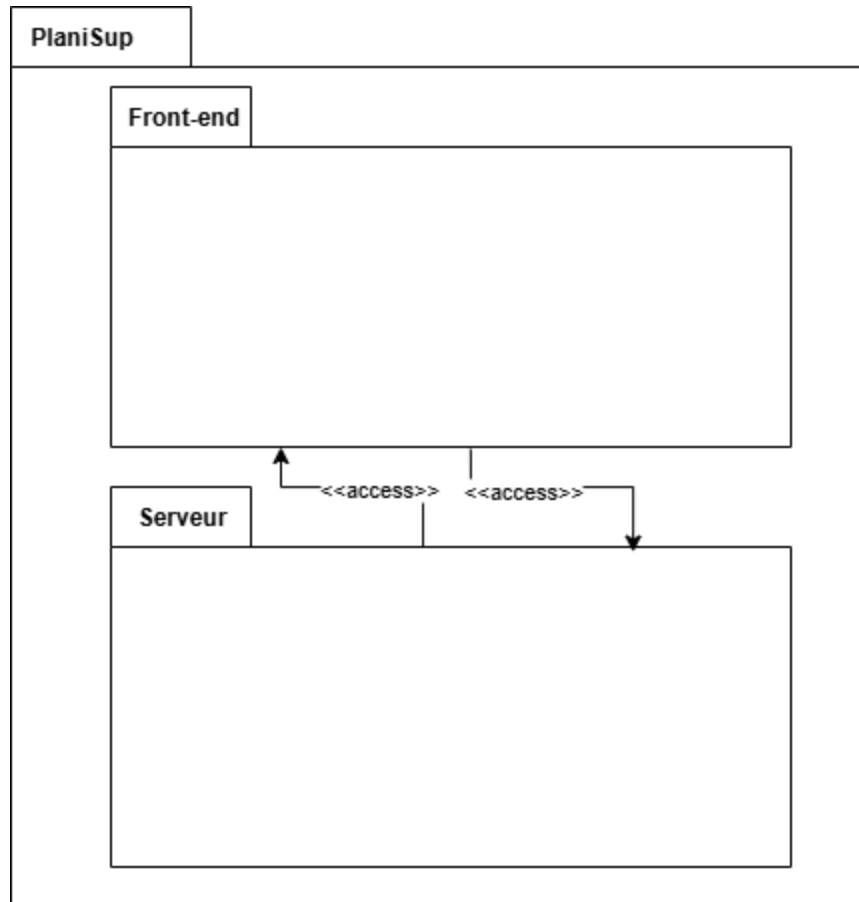


Figure 4.1 - Diagramme de paquetages de l'application globale

| Paquetage        | Fonction                                                                                                 |
|------------------|----------------------------------------------------------------------------------------------------------|
| Authentification | Se charge du compte de l'utilisateur et sa connexion                                                     |
| Étudiant         | Se charge des choix de programmes et la gestion de son plan d'études                                     |
| Administrateur   | Se charge de gérer les rôles des autres utilisateurs, de gérer les programmes et de configurer les cours |
| Employé          | Se charge d'approuver les plans d'études                                                                 |
| Liste de Plan    | Se charge des listes de tous les plans d'études                                                          |

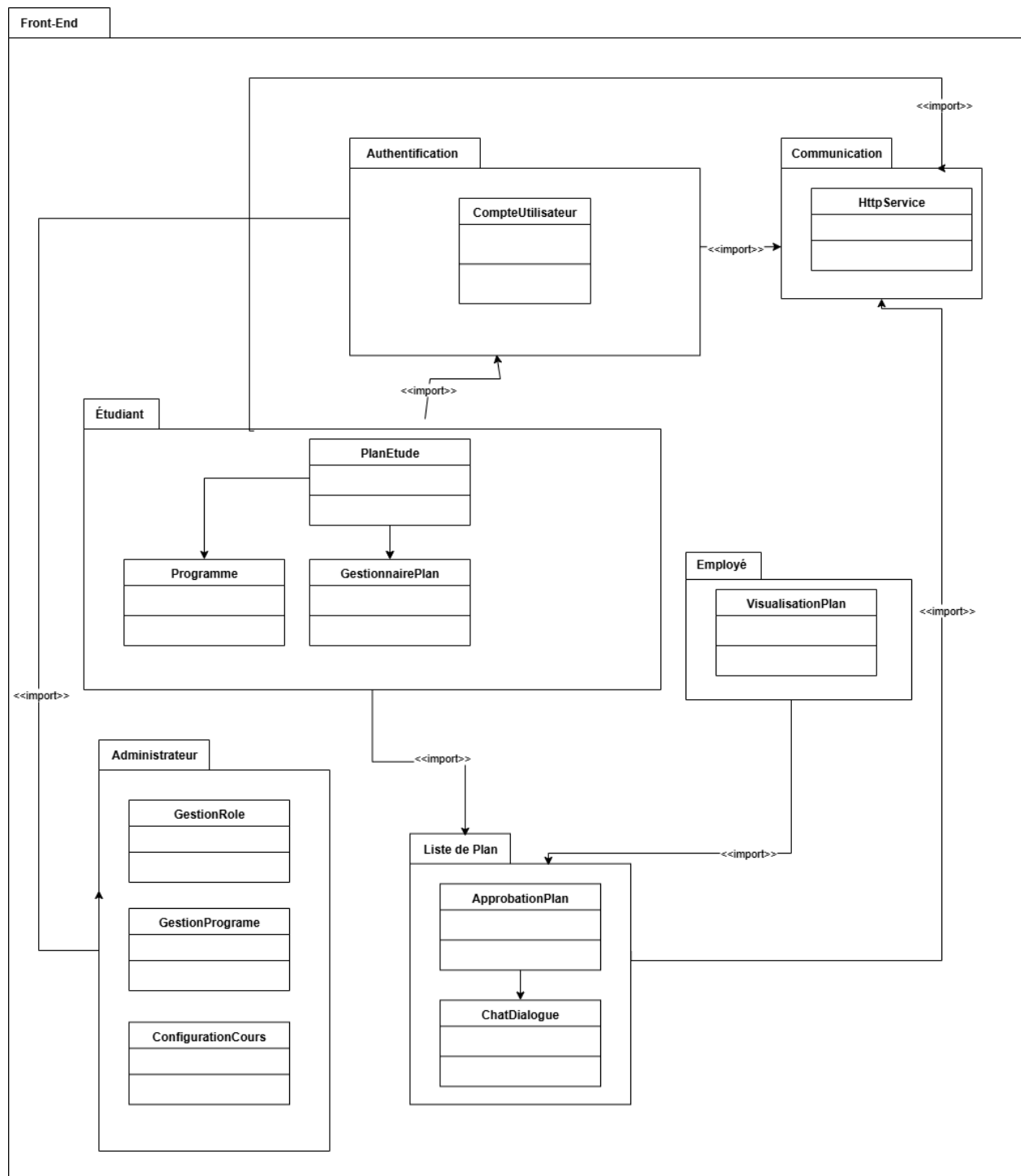


Figure 4.2 - Diagramme de paquetages du Front-End

| Paquetage           | Fonction                                         |
|---------------------|--------------------------------------------------|
| Gestion Utilisateur | Se charge de la gestion des comptes utilisateurs |
| Gestion Discussion  | Se charge des canaux de discussion               |
| Communication       | Se charge de la communication hors serveur       |



|                       |                                                 |
|-----------------------|-------------------------------------------------|
| Base de données       | Se charge de la gestion des données             |
| Gestion Plan d'études | Se charge de tout le processus du plan d'études |

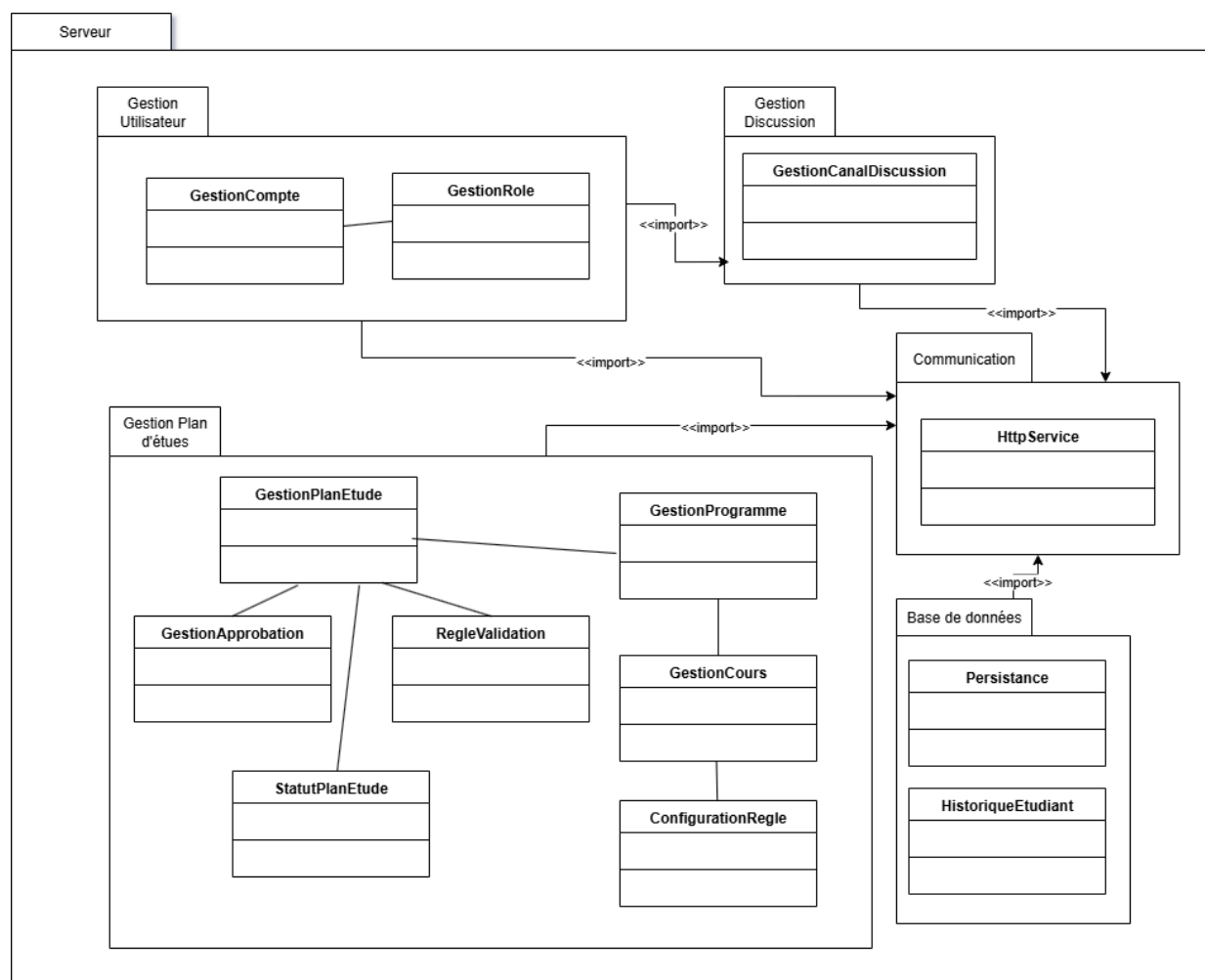


Figure 4.3 - Diagramme de paquetages du Serveur

## 5. Vue des processus

Cette section contient les diagrammes de séquence de PlaniSup.

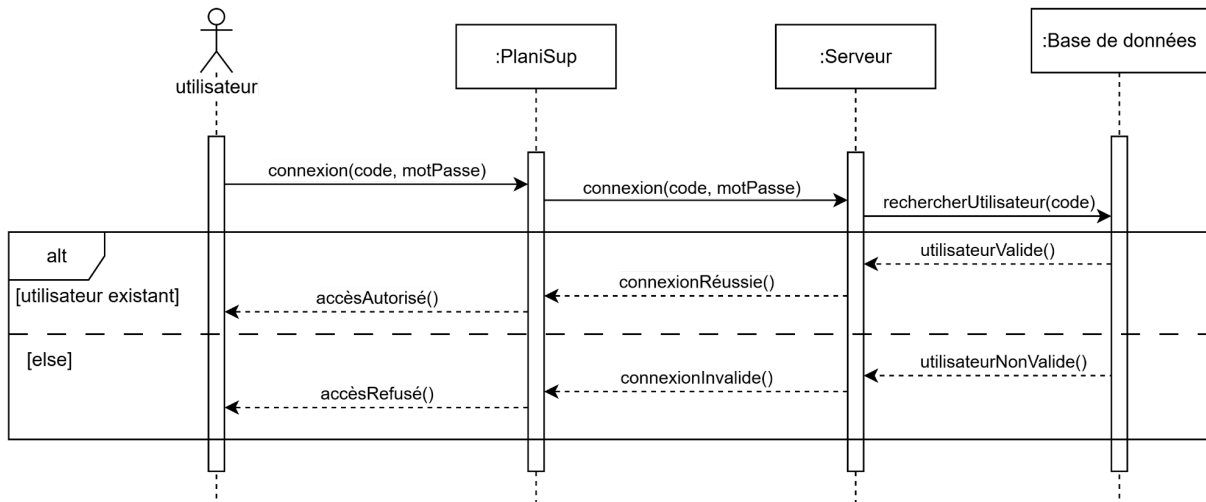


Figure 5.1 - Diagramme de séquence pour l'authentification

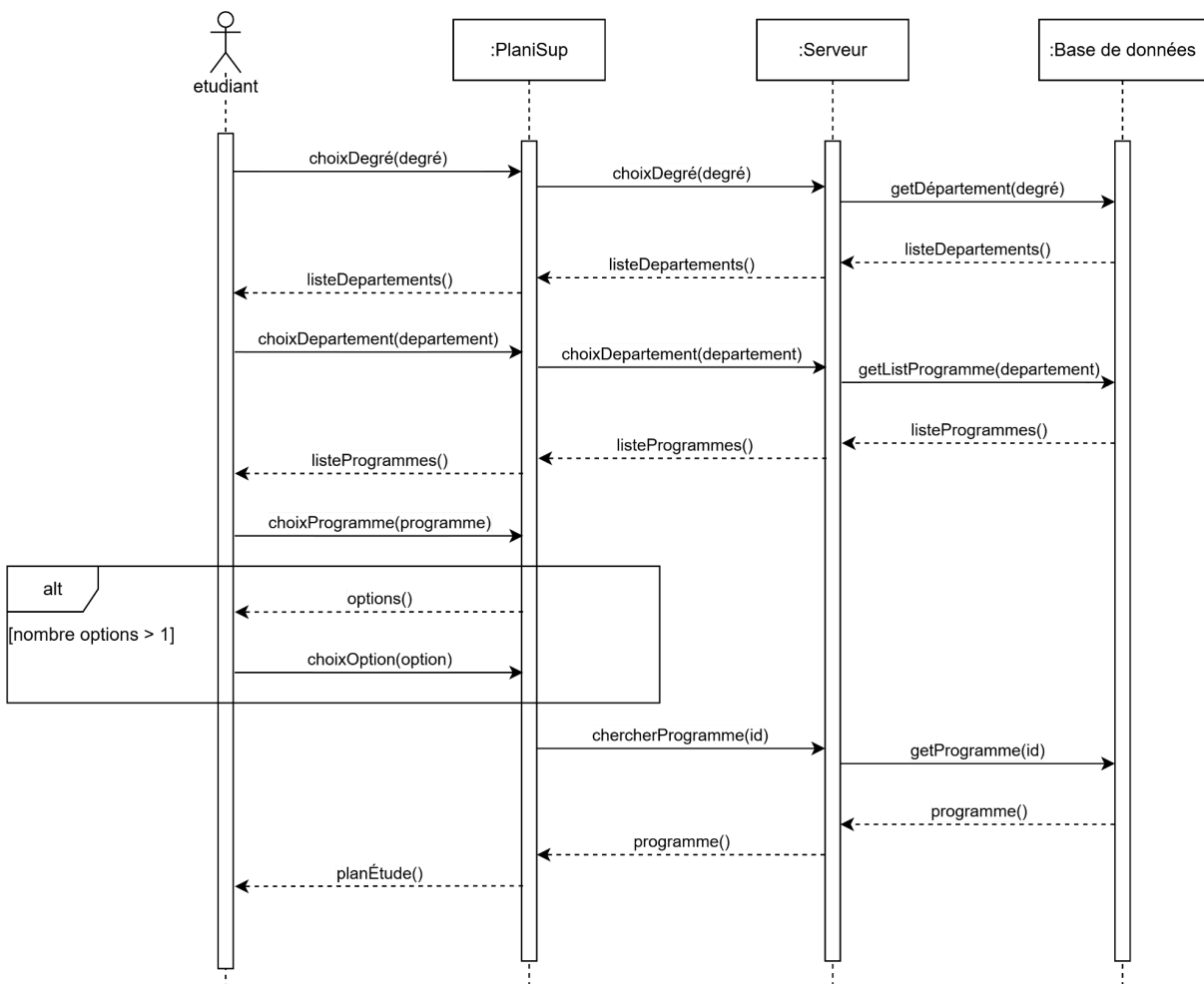


Figure 5.2 - Diagramme de séquence pour visualiser un plan d'études

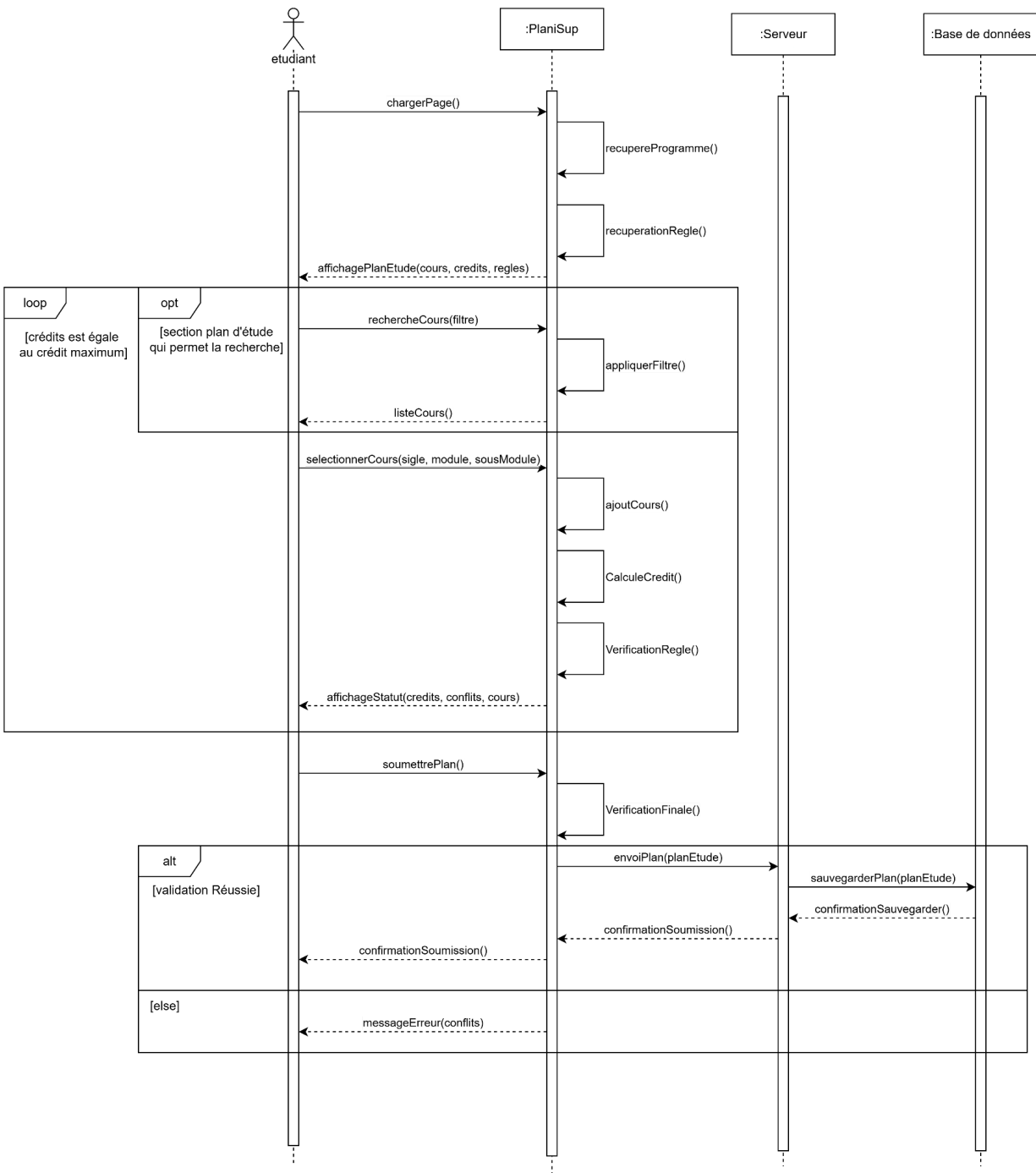


Figure 5.3 - Diagramme de séquence pour soumettre un plan d'études

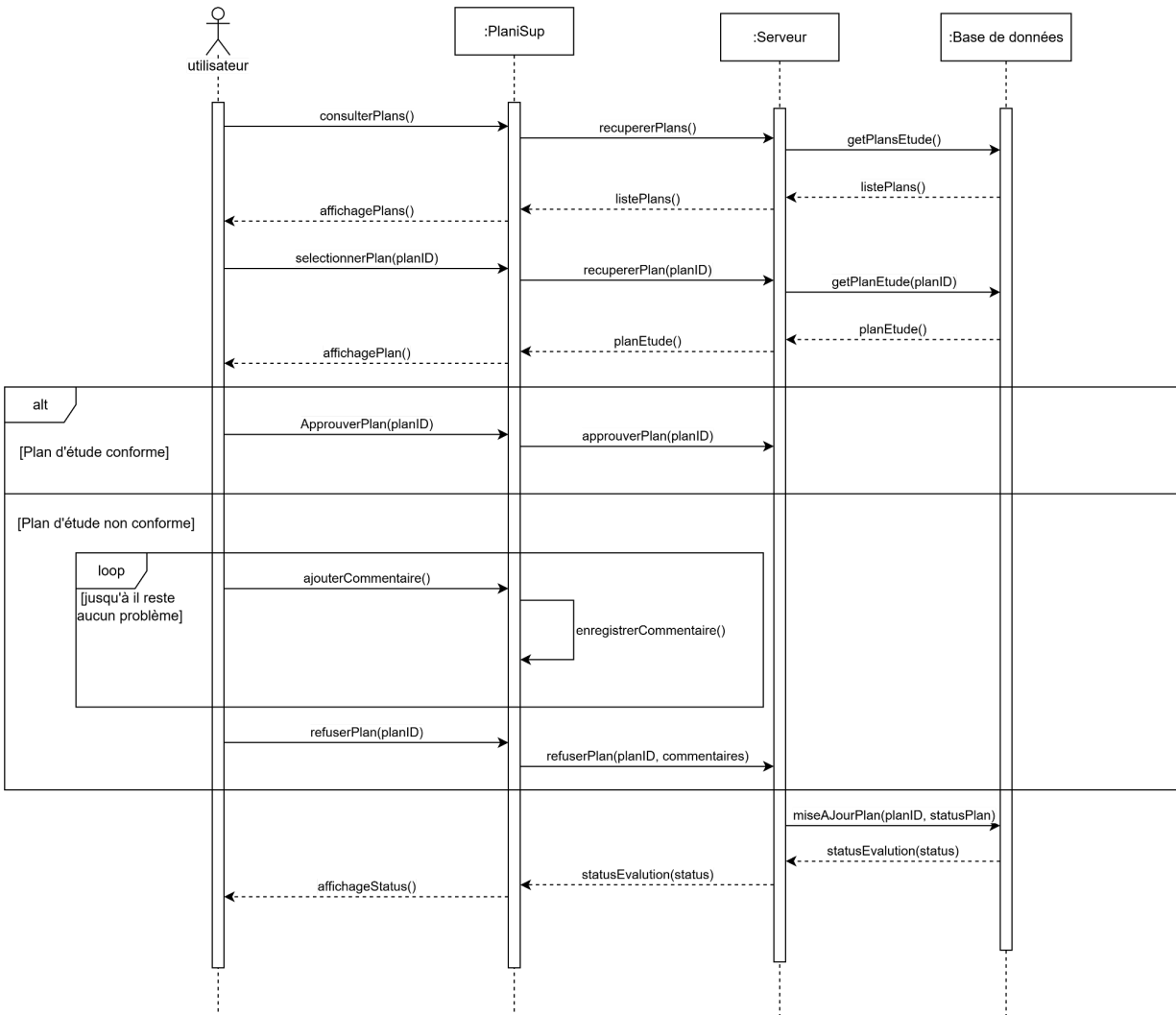


Figure 5.4 - Diagramme de séquence pour approuver/rejeter un plan d'études

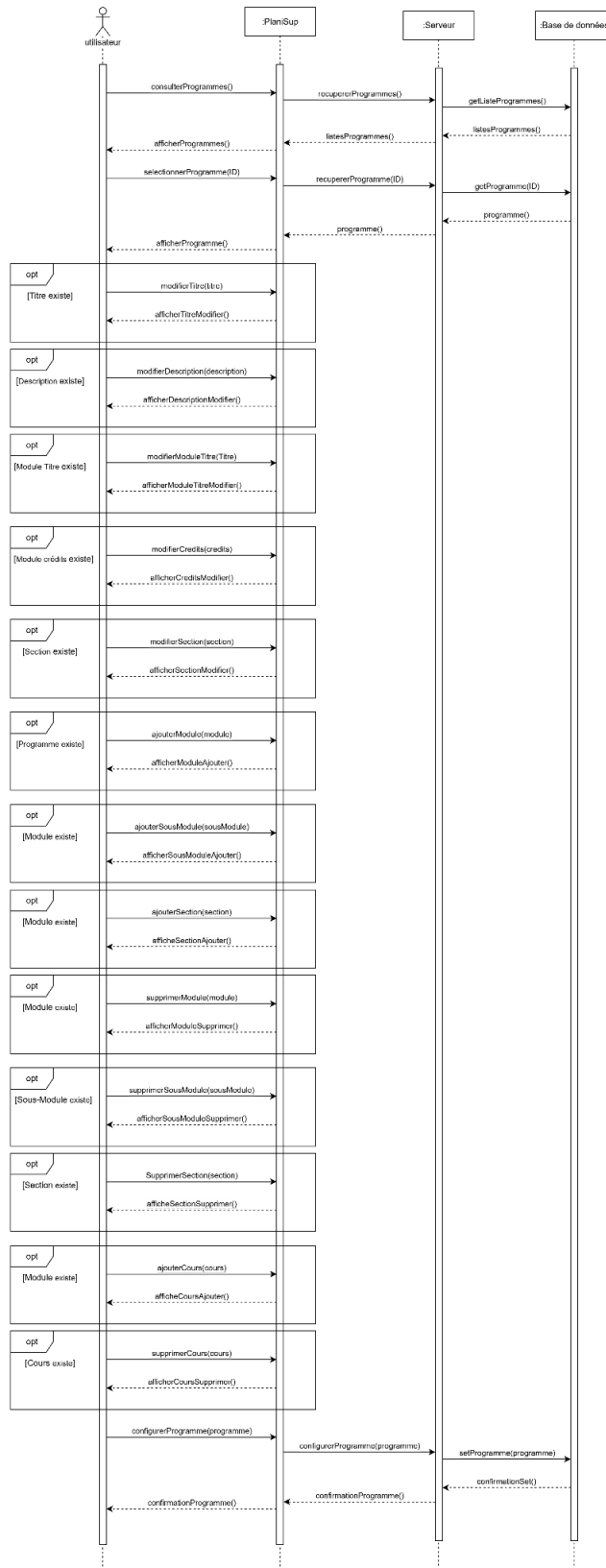


Figure 5.5 - Diagramme de séquence pour configurer les programmes comme administrateur

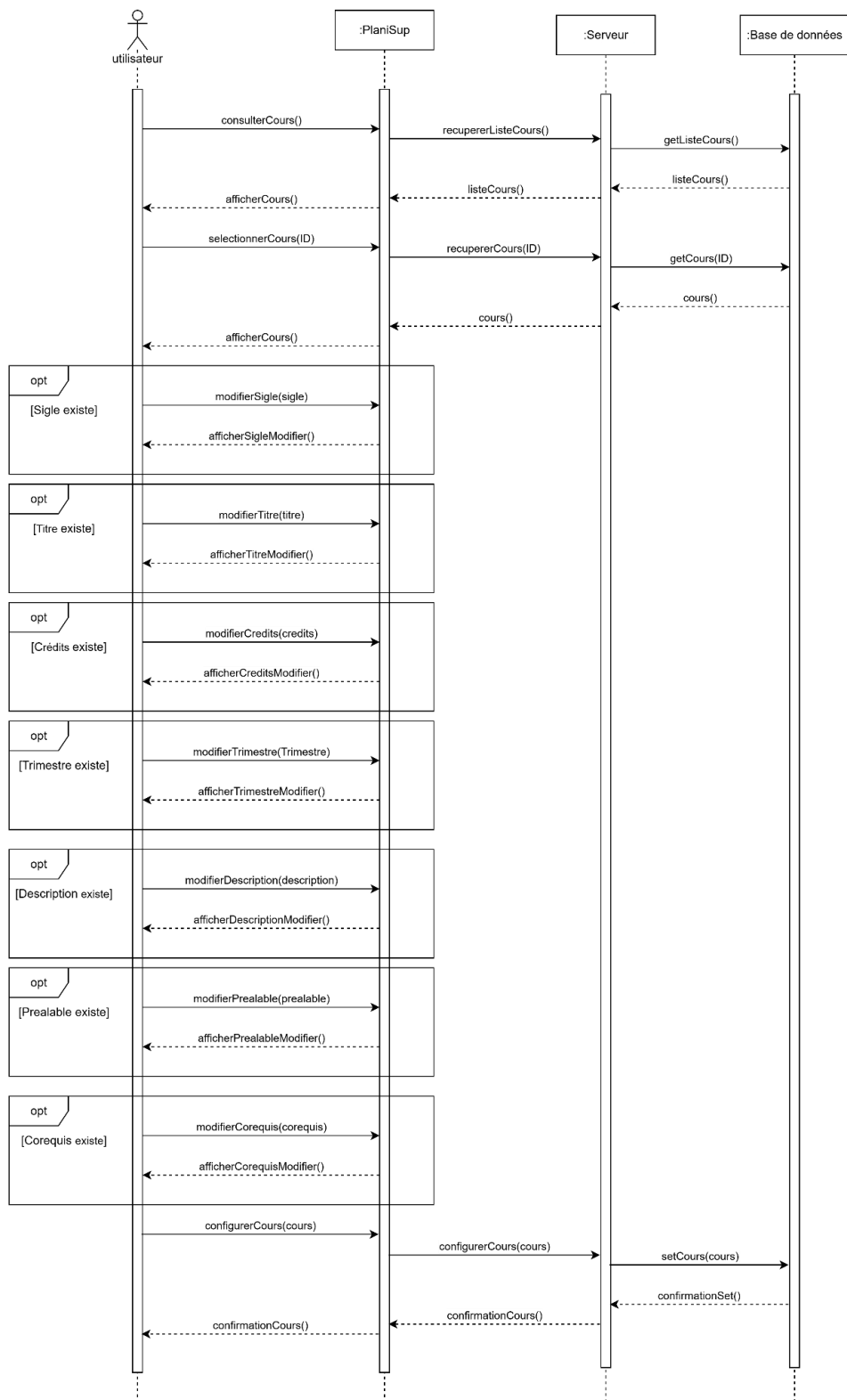


Figure 5.6 - Diagramme de séquence pour configurer les cours comme administrateur

## 6. Vue de déploiement

Cette section présente le déploiement de PlaniSup. Le diagramme illustre la distribution des composants logiciels sur l'infrastructure matérielle, les protocoles de communication utilisés et les liens entre les différents éléments du système.

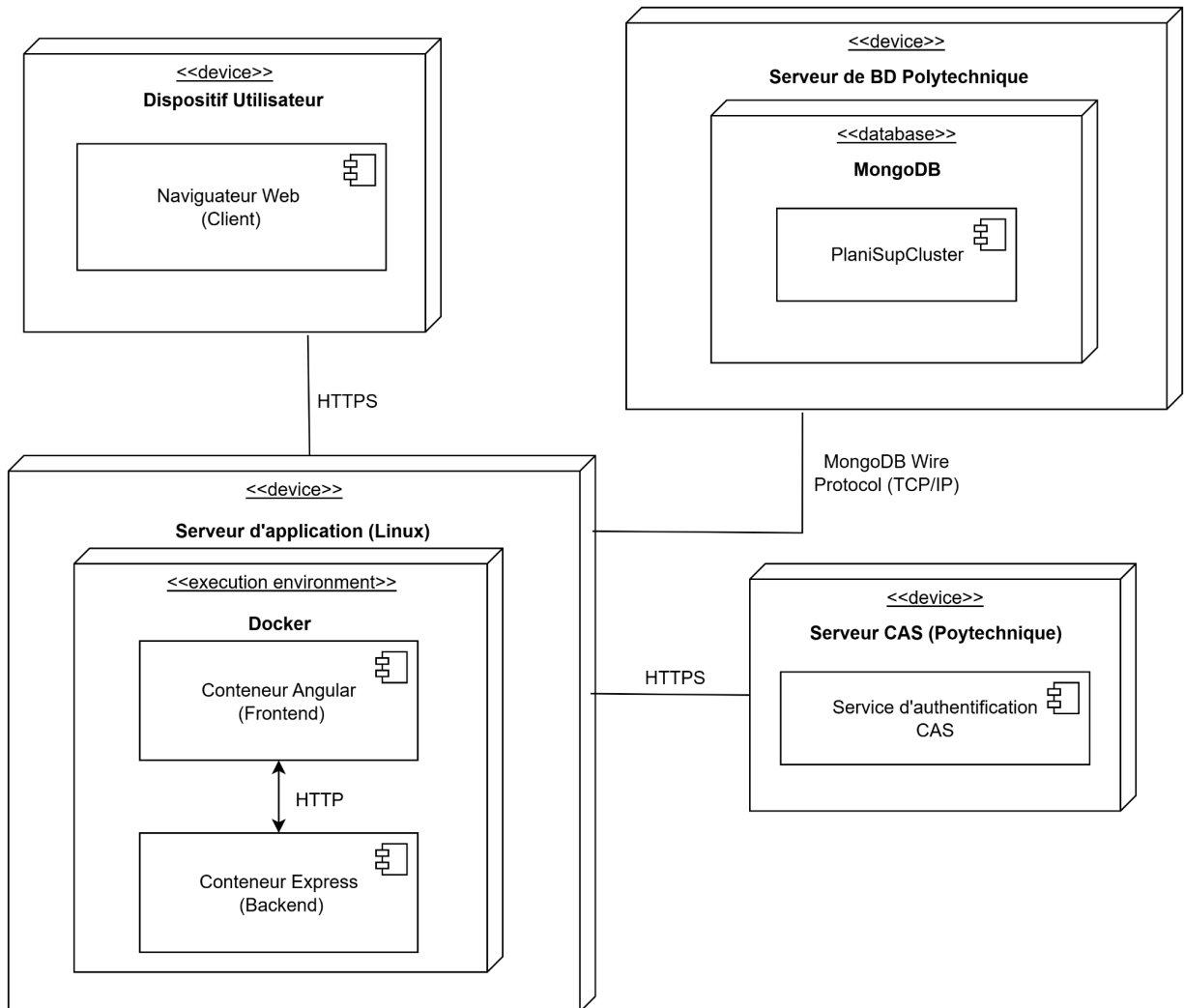


Figure 6 - Diagramme de déploiement

L'architecture de déploiement de PlaniSup s'articule autour de quatre composants principaux.

Le serveur d'application s'exécute sur un serveur Linux fourni par le Service informatique de Polytechnique Montréal. L'architecture repose sur Docker comme environnement d'exécution conteneurisé. Deux conteneurs distincts sont déployés : le conteneur Angular héberge l'application frontend qui sert l'interface utilisateur, tandis que le conteneur Express contient le serveur backend. Cette séparation en conteneurs facilite le déploiement indépendant de chaque couche.

La base de données MongoDB est également hébergée sur l'infrastructure de Polytechnique Montréal. Le cluster PlaniSupCluster stocke l'ensemble des données métier de l'application : les structures de programmes avec leurs

règles de validation configurables, le catalogue complet des cours, les plans d'études des étudiants, l'historique des approbations et commentaires, ainsi que les informations des utilisateurs et leurs rôles. MongoDB a été choisi pour sa flexibilité dans la gestion de structures de données évolutives, permettant d'adapter facilement les règles académiques qui varient selon les départements et les types de programmes sans modification du schéma.

Le serveur CAS (Central Authentication Service) de Polytechnique assure l'authentification centralisée des utilisateurs. Ce service externe, géré par l'établissement, permet aux utilisateurs de s'authentifier avec leurs identifiants institutionnels.

Les utilisateurs accèdent à PlaniSup via leur navigateur web. La communication entre le navigateur et le serveur d'application s'effectue en HTTPS, assurant le chiffrement des données académiques sensibles pendant le transit. À l'intérieur de l'environnement Docker, le conteneur Angular communique avec le conteneur Express via HTTP d'une API REST. Le backend Express se connecte à MongoDB via le MongoDB Wire Protocol sur TCP/IP, qui transporte les requêtes et réponses entre l'application et la base de données. L'authentification avec le serveur CAS s'effectue également en HTTPS.

## **7. Taille et performance**

Cette section décrit les caractéristiques de taille et de performance du système PlaniSup qui influencent directement les décisions architecturales et de conception. Ces métriques visent à garantir une expérience utilisateur fluide, une utilisation efficace des ressources et le respect des contraintes d'hébergement institutionnelles.

### **7.1. Application Web (Client)**

L'application web, développée avec Angular, doit offrir des performances optimales sur les divers navigateurs. Le bundle de production de l'application Angular ne doit pas dépasser 10 Mo afin de garantir des temps de chargement initiaux rapides conformément à l'exigence de performance demandant un chargement de la page d'accueil en moins de 2 secondes. Pendant une utilisation standard, l'application cliente ne doit pas consommer plus de 150 Mo de mémoire RAM afin de maintenir la réactivité de l'interface lors de la manipulation interactive des plans d'études. Conformément aux exigences du SRS, 95 % des actions de l'utilisateur, telles que la navigation, la validation, ou la recherche, doivent avoir un temps de réponse perçu inférieur à 2 secondes tandis que les opérations critiques comme la validation automatique d'un plan d'études doivent s'exécuter en moins de 5 secondes.

### **7.2. Serveur d'application (Backend)**

Le serveur sera développé avec Express, doit gérer les requêtes de tous les clients et assurer l'intégrité des processus métier. Le système doit supporter au moins 500 utilisateurs connectés simultanément sans dégradation notable des performances ce qui correspond à la population étudiante visée aux cycles supérieurs. En conditions de charge nominale, l'instance du serveur ne doit pas utiliser plus de 20 % d'un cœur de CPU et ne pas dépasser 500 Mo de RAM pour rester efficace sur l'infrastructure serveur de Polytechnique Montréal. De plus, le serveur doit traiter 95 % des requêtes API incluant les opérations GET, POST et PUT en moins de 500 ms pour respecter les contraintes de réactivité globale du système.



### **7.3. Base de données**

La base de données MongoDB est utilisée pour la persistance de l'ensemble des données métier incluant les programmes, les cours, les plans d'études, les utilisateurs et les historiques d'approbation. La taille initiale de la base de données est estimée à moins de 100 Mo comprenant le catalogue de cours et la structure des programmes avec une croissance prévisionnelle d'environ 50 Mo par année en fonction du nombre de nouveaux plans d'études et d'utilisateurs. Les requêtes fréquentes, telles que la récupération du catalogue de cours ou la validation d'un plan d'études doivent s'exécuter en moins de 100 ms pour contribuer aux temps de réponse globaux exigés. La base de données doit être configurée pour assurer une disponibilité conforme à l'exigence de fiabilité de 99,9 %.

### **7.4. Performance réseau**

Les communications entre le client et le serveur s'effectuent via des API par HTTP. La conception de l'application doit tolérer une latence réseau allant jusqu'à 200 ms sans impact significatif sur l'expérience utilisateur en privilégiant des requêtes légères et une gestion asynchrone des actions. Le volume de données échangées pour des opérations courantes, comme l'affichage d'un plan d'études, doit être optimisé et ne pas dépasser 1 Mo par requête afin de minimiser le temps de transfert en particulier sur des connexions moins stables.